

Parcial 1 (M1) - Aplicaciones Web 1

Alumno: Mateo Soria

Carrera: Analista de Sistemas - IES Siglo 21

Docente: Andrés Senn

Parcial n.º: 1

Tema elegido: 1) Arquitectura semántica HTML5 y modelo de caja CSS

Marco teórico

HTML: marcado semántico y jerarquía de contenidos

El marcado semántico consiste en elegir cada etiqueta por el significado de su contenido y no por su aspecto visual. HTML5 incorporó contenedores con un rol definido: `<header>` agrupa el contenido introductorio, `<nav>` los enlaces de navegación, `<main>` el contenido central (único por página), `<section>` un bloque temático con su propio encabezado, `<article>` una pieza independiente y `<footer>` el cierre con datos de autoría o contacto. A diferencia de `<div>`, que es un contenedor genérico sin significado, estas etiquetas generan regiones de referencia (landmarks ARIA: banner, navigation, main, contentinfo) que los lectores de pantalla anuncian y permiten recorrer con atajos, cumpliendo el criterio WCAG 1.3.1 (Información y relaciones). Los motores de búsqueda también usan esta jerarquía para identificar el contenido relevante, lo que favorece el SEO.

CSS: modelo de caja alternativo y reseteo global

Según el modelo de caja, todo elemento se representa como una caja formada por contenido, padding, border y margin. Con el valor por defecto, `box-sizing: content-box`, `width` y `height` definen solo el área de contenido, por lo que el padding y el borde se suman: una caja de 300 px de ancho con 20 px de padding y 10 px de borde ocupa 360 px. Con `border-box`, el ancho declarado incluye padding y borde: la caja mide 300 px y el contenido se reduce a 240 px. `box-sizing` no es una propiedad heredable, por eso el reseteo global la declara en `html` y la propaga con `*`, `*::before`, `*::after` { `box-sizing: inherit`; }, además de anular los márgenes por defecto de `html` y `body`.

Interacción entre HTML y CSS

HTML aporta la estructura y el significado: cada contenedor semántico genera una caja de bloque en el flujo normal. CSS decide cómo se dimensionan y distribuyen esas cajas. El reseteo con `border-box` vuelve predecible el tamaño de cada región semántica, sin agregar `<div>` envoltorios solo para corregir medidas, y así el marcado se mantiene limpio y accesible.

Fuentes de consulta

1. Senn, A. *Aplicaciones Web I*. Editorial IES Siglo 21. SP2 (Compaginación HTML, accesibilidad) y SP3 (modelo de caja).
2. MDN Web Docs. *box-sizing*. <https://developer.mozilla.org/es/docs/Web/CSS/box-sizing>
3. MDN Web Docs. *Elementos HTML (referencia) - Seccionamiento de contenido*. <https://developer.mozilla.org/es/docs/Web/HTML/Element>
4. W3C WAI. *ARIA Landmarks Example*. <https://www.w3.org/WAI/ARIA/apg/patterns/landmarks/>
5. W3C. *WCAG 2.1 - Criterio 1.3.1 Información y relaciones*. <https://www.w3.org/TR/WCAG21/#info-and-relationships>

Demostración codificada

Archivos: `src/index.html` y `src/recursos/estilos/estilos.css`. Se ejecuta con `pnpm install` y `pnpm dev` (five-server, raíz en `src`). El body se divide en tres regiones independientes: header, main y footer. La tipografía se carga de forma local con `@font-face` desde `recursos/estilos/fuentes` (Liberation Sans, equivalente libre de Arial) y la paleta usa tonos de azul y blanco.

1. Estructura semántica (antes vs. después)

La página está maquetada solo con header, nav, main, section, article y footer, sin ningún div. Cada section se vincula a su título con `aria-labelledby` y el nav tiene `aria-label`. Dentro de la página se muestra el mismo esquema escrito con `div` (antes) y con etiquetas semánticas (después).

2. content-box (por defecto) vs. border-box (modificado)

Dos cajas con las mismas declaraciones: `width: 300px; height: 150px; padding: 20px; border: 10px`.

box-sizing	Cálculo del ancho	Tamaño final renderizado	Área de contenido
content-box	$300 + 20 \times 2 + 10 \times 2$	360×210 px	300×150 px
border-box	300 (incluye padding y borde)	300×150 px	240×90 px

Los valores se verificaron en el navegador (DevTools > Computed): `offsetWidth` de 360 px y 300 px respectivamente. Con `background-clip: content-box` el azul claro pinta solo el área de contenido y se ve cuánto espacio queda en cada caso.

3. ¿box-sizing es heredable?

No. Para probarlo se usan dos hijos con `width: 100%`, 16 px de padding y 4 px de borde:

- **Caso 1:** hereda `border-box` gracias al reseteo con `inherit` y encaja exacto en su contenedor.
- **Caso 2:** usa `box-sizing: unset`, que equivale a `inherit` si la propiedad es heredable y a `initial` si no lo es. El valor computado es `content-box` aunque el padre tenga `border-box`, lo que demuestra que la propiedad no se hereda sola. El hijo desborda 40 px ($16 \times 2 + 4 \times 2$).

Reseteo global aplicado

```
html {
  box-sizing: border-box;
}

*,
*::before,
*::after {
  box-sizing: inherit;
}

html,
body {
  margin: 0;
  padding: 0;
}
```

Etiquetas semánticas

header y nav

header agrupa el contenido introductorio y nav identifica los enlaces de navegación. Los lectores de pantalla los anuncian como regiones de referencia (landmarks).

main, section y article

main contiene el contenido central, section lo divide por temas y article encapsula piezas independientes.

footer

footer cierra la página o una sección con datos de autoría, contacto o enlaces.

Antes vs. después

Visualmente las dos versiones pueden verse iguales, pero solo la segunda le comunica al navegador, a los lectores de pantalla y a los buscadores qué es cada parte.

Antes: solo div (sin significado)

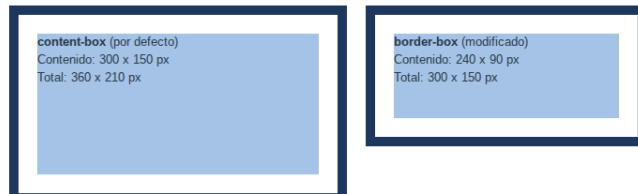
- `div class="cabecera"`
- `div class="menu"`
- `div class="contenido"`
- `div class="tema"`
- `div class="pie"`

Después: marcado semántico

- `header`
- `nav`
- `main`
- `section`
- `footer`

Modelo de caja: content-box vs. border-box

Las dos cajas tienen las mismas declaraciones (`width`, `height`, `padding` y `border`). Solo cambia `box-sizing`. El azul claro marca el área de contenido.



¿box-sizing se hereda?

No se hereda por defecto. Por eso el reseteo usa `inherit` para pasar el valor de `html` a todos los elementos.

Caso 1: con reseteo (hereda border-box)

width: 100% - encaja en el contenedor

Caso 2: sin herencia (box-sizing: unset)

width: 100% - desborda el contenedor